

Hashing

IT-SICHERHEIT / SEMESTER 5 UND 6



1

AGENDA

- 01 GRUNDLAGEN
- 02 MERKLE-DAMGÅRD-CONSTRUCTION
- 03 MAC
- 04 REGENBOGENTABELLEN

2

01

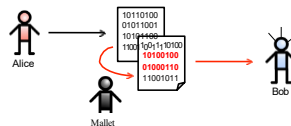
Grundlagen

3

IT-SICHERHEIT / SEMESTER 5 UND 6

Hash-Funktionen zur Integritätssicherung

- **Ziel:** Sicherstellen, dass Manipulationen an einer übertragenen Nachricht erkannt werden.



- **Beispiel:** Softwareverteilung



Current Sources

This tarball contains the latest Linux sources.

aircrack-ng-1.1.tar.gz

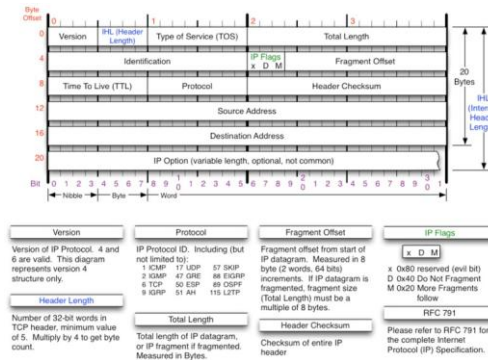
SHA1: 16eed1a8cf06eb8274ae382150b56589b23adf77

MD5: f7a24ed8fad122c4187d06bfd6f998b4

4

Hash-Funktionen zur Integritätssicherung

- Prüfsummen dienen der **Erkennung von (unbeabsichtigten) Übertragungsfehlern**, z.B. beim IPv4-Header:



- Kryptographische Prüfsummen sollen auch **absichtliche Manipulationen erschweren**



[Quelle: Kapitel 9: Kryptographische Hash-Funktionen, Irz, IT-Sicherheit, WS 21/22, Helmut Reiser]

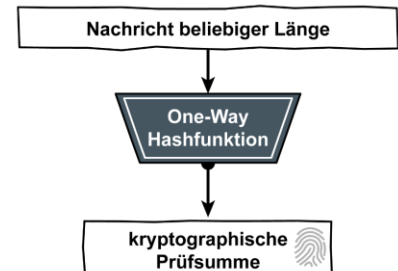
tgm | Technologisches Gewerbemuseum | Höhere technische Bundes-Lehr- und Versuchsanstalt

5

5

Grundlagen

- Die Digitale Signatur entspricht einer Operation mit **einem Public-Key-Verfahren und ist daher sehr rechenintensiv**.
- Um dem Aufwand zu vermindern, signiert man nicht die gesamte Information mit dem Public-Key-Verfahren, sondern erstellt eine **Prüfsumme als "Konzentrat" der Nachricht**, das digital signiert wird.
- Auf eine Nachricht, deren Länge variabel ist, wird eine sogenannte **One-Way-Hashfunktion** angewendet, die eine **kryptographische Prüfsumme (Hashwert)** fester kurzer Länge als Ergebnis erzeugt.
- Zu den besonderen Eigenschaften von One-Way-Hashfunktion gehört, dass die Berechnung des **Funktionswertes einfach** ist, während es aber praktisch unmöglich ist, systematisch einen Wert zu finden, der dieselbe kryptographische Prüfsumme ergibt.



Ein Beispiel, das den Hash-Algorithmus SHA-256 verwendet:

„fidentity“ → Hash-Wert:
5a25417fa112c7bc591d7a7...

„fidentity AG“ → Hash-Wert:
e753c1e761847c0f88b2...



[Quelle: Cyber Sicherheit; Pohlmann, Norbert; Springer Fachmedien Wiesbaden; 2. Auflage 2022, Kapitel 2 Kryptographie]

tgm | Technologisches Gewerbemuseum | Höhere technische Bundes-Lehr- und Versuchsanstalt

6

6

Hashing im Alltag

- **Passwörter schützen:** Hashing wandelt Passwörter in eindeutige Werte um, die selbst bei einem Datenbank-Leck nicht im Klartext sichtbar sind. Stattdessen werden nur die berechneten Hash-Werte gespeichert, wodurch ein hohes Mass an Sicherheit gewährleistet wird.
- **Blockchain sichern:** Hashing stellt sicher, dass jede Transaktion in einer Blockchain unveränderlich bleibt. Jede Transaktion wird durch einen **Hash-Wert** dokumentiert, der mit vorherigen Transaktionen verknüpft ist. Dadurch wird die Integrität der gesamten Blockchain garantiert.
- **Digitales Signieren:** Beim digitalen Signieren wird der Hash-Wert eines Dokuments berechnet und in der Signatur verschlüsselt. Der Empfänger entschlüsselt den Wert und vergleicht ihn mit einem neu berechneten Hash-Wert. Stimmen beide Werte überein, ist das Dokument unverändert und die Integrität gewährleistet.

7

„Hash“ – ein einfaches Summenbeispiel

- Wird aus dem Gesamttext errechnet.
- Vergleichsbeispiel einer – **primitiven – Hash-Funktion:** Jeder Buchstabe wird durch seine **Position im Alphabet** ersetzt, am Schluss werden diese Zahlen zusammengezählt:

Buchstabe	S	C	H	M	E	T	T	E	R	L	I	N	G	Summe
Position im Alphabet	19	3	8	13	5	20	20	5	18	12	9	14	7	153

- Natürlich kommen wesentlich komplexere Verfahren zum Einsatz. Z.B. **SHA-256**, womit Hash-Werte mit einer Länge von 256 Bit erzeugt werden - üblicherweise als 64-stellige Hexadezimal-Zahl ausgedrückt werden. Der Hash-Wert für das Wort „Schmetterling“ z.B. lautet dann:

d7e3dabc2c95c4c440ee57fb2883188e7f46a9cf51e94674f0 e80f7d6db092c4

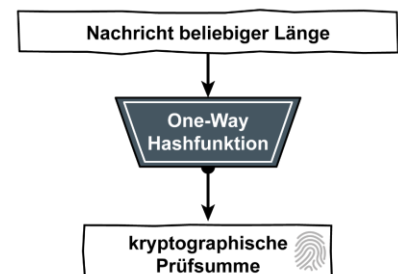
8

„CRC32“ Prüfsumme

- CRC steht für **Cyclic Redundancy Check**, also 'zyklische Redundanzprüfung' und ist ein Verfahren zur Bestimmung eines **Prüfwerts für Daten**, um Fehler bei der Übertragung oder Speicherung erkennen zu können.
- Das Verfahren wurde 1961 von W. Wesley Peterson entwickelt. Der Hash ist 32 bit, also 4 Byte lang.
- CRC ist so ausgelegt, dass **Fehler bei der Übertragung der Daten**, wie sie beispielsweise durch Rauschen auf der Leitung verursacht werden könnten, **mit hoher Wahrscheinlichkeit entdeckt** werden.
- Zum Beispiel werden **Datenübertragungen über Ethernet, sowie die meisten Festplatten-Übertragungen** mit CRC-Verfahren geprüft.
- Die Berechnung des CRC-Werts beruht auf **Polynomdivision**: Die Folge der zu übertragenden Bits wird als binäres Polynom betrachtet.

Eigenschaften

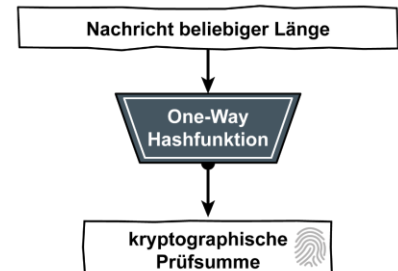
- **H ist eine öffentliche bekannte Einwegfunktion**
 - Muss allen bekannt und verfügbar sein
 - öffentlichen Diskussion von Experten standgehalten
- **$h = H(M)$, h ist ein eindeutiger "Fingerabdruck" von M (Hashwert)**
 - Eingabe **M** kann beliebig lang sein
 - Hashwert **h** hat eine feste Länge, z.B. 256 Bit
- **$H(M)$ ist eine One-Way Funktion (Einwegfunktion)**
 - **$H(M)$ ist einfach zu berechnen**, bei gegebenem M
 - Mit gegebenem h, ist es schwer (praktische unmöglich) M zu berechnen, sodass $M = H^{-1}(hM)$ ist.



Eigenschaften

$H(M)$ ist kollisionsresistent

- Mit gegebenem M , ist es schwer (praktisch unmöglich) eine weitere Nachricht M' zu finden, sodass $H(M) = H(M')$!
- Zwei verschiedene digitale Dokumente (Nachrichten), die denselben Hashwert abbilden werden, bilden eine Kollision!
- Die Existenz von Kollisionen ist unvermeidbar.
- Diese ist aber nur eine theoretische Aussage.
- Bei praktischen Anwendungen kommt es nur darauf an, dass es, wie oben verlangt, praktisch unmöglich ist, Kollisionen zu finden.



11

Arbeitsweise

Das Original

In Xanadu did Kubla Khan
A stately pleasure-dome decree:
Where Alph, the sacred river, ran
Through caverns measureless to man
Down to a sunless sea

Hashfunktion

Hashwert

a89d e23f ede8

Die veränderte Kopie

In Xanadu did Napoleon
A stately pleasure-dome decree:
Where Alph, the sacred river, ran
Through caverns measureless to man
Down to a sunless sea

Hashfunktion

Hashwert

38fe 38aa 9c2d

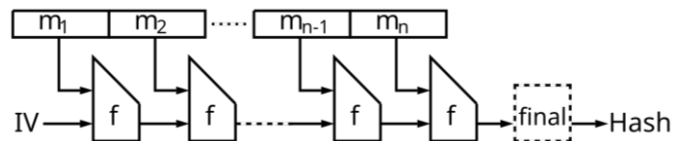
Minimaler Unterschied

Großer Unterschied

12

Konstruktion kryptographischer Hash-Funktionen

- Folge von **Kompressionsfunktionen** f
- Nachricht m wird in **Blöcke** m_i mit **fester Länge** y zerlegt
- Hash-Verfahren wird mit **Initialisierungswert** IV vorbelegt
- Letzter Block m_n muss ggf. auf vorgegebene Länge y „**aufgefüllt**“ werden (**Padding**)
- Als Kompressionsfunktion f können verwendet werden:
 - Hash-Funktionen auf der Basis symmetrischer Blockchiffren
 - Dedizierte Hash-Funktionen



Passwort-Salting

- Um die Vorhersehbarkeit einer Hash-Funktion zu minimieren, verwenden Programmierer Salting.
- Salting ist eine Technik, die die vorhersehbare Hash-Funktion nimmt und etwas zusätzliches „Gewürz“ – daher der Begriff „Salz“ – in Form von Unvorhersehbarkeit hinzufügt.
- Ein Salz ist eine kurze Reihe zufälliger Zeichen, die vor dem Hashen zu jedem Passwort hinzugefügt wird.

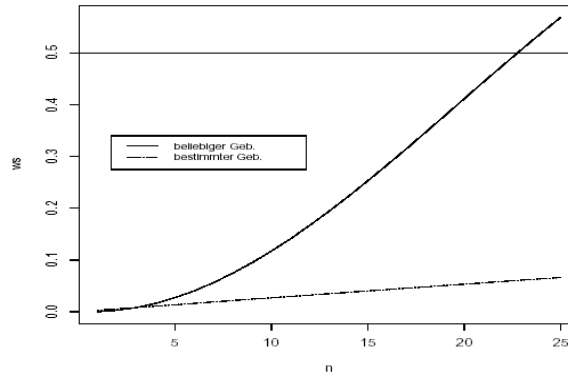
Username:	String to be hashed:	Hashed value = SHA256
user1	password123	EF92B778BAFE771E89245B89ECBC08A44A4E166C06659911881F383D4473E94F
user2	password123	EF92B778BAFE771E89245B89ECBC08A44A4E166C06659911881F383D4473E94F

Username:	Salt value:	String to be hashed:	Hashed value = SHA256 (Password + Salt value)
user1	D;%yL9TS:5PaIS/d	password123D;%yL9TS:5PaIS/d	9C9B913EB1B6254F4737CE947EFD16F16E916F9D6EE5C1102A2002E48D4C88BD
user2	%<~<U(jLezy4j>*	password123)%<~<U(jLezy4j>*	6058B4EB46BD6487298B59440EC8E70EAE482239FF2B4E7CA69950DFBD5532F2

Das Geburtstags-Paradox

Wie viele zufällig ausgewählte Menschen braucht man, damit mit $p > 0,5$ mindestens 2 am gleichen Tag Geburtstag haben?

Die Entwicklung der Wahrscheinlichkeiten



Das Geburtstags-Paradox

- **Aufgabe:** Ein Angreifer will ein geändertes Dokument erzeugen, das denselben Hashwert liefert, da nur der Hashwert bei einer digitalen Signatur von einem elektronischen Dokument signiert wird.
- **Problem:** Wenn eine Hashfunktion H Nachrichten auf Komprimierte einer Länge von 60 Bit abbildet, dann braucht ein Angreifer rein statistisch „nur“ 2^{30} verschiedene Eingabennachrichten, um eine Kollision zu finden (in Texten z.B. lange Artikelnummern).

Anforderung an Hashfunktionen

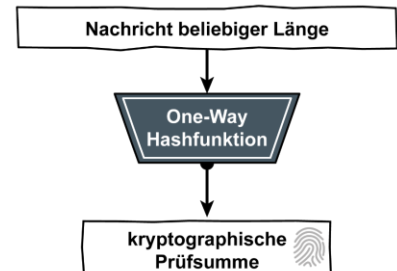
- Hashfunktionen H sollten einen längeren Hashwert h haben als Schlüssellängen bei symmetrischen Verschlüsselungsverfahren, z.T. mindestens 160 Bit!

Übersicht

Hashfunktionen sind praktisch unumkehrbare Komprimierungsfunktionen

Typische Vertreter:

- MD5, SHA, ...
- RIPEMD = RIPE Message Digest
1992 innerhalb des EU-Projekts RIPE entwickelt;
- SHA-3 neuer NIST-Standard



Zusammenfassung der Anforderungen an die Hash-Funktion

- Kann auf eine Datei beliebiger Länge angewandt werden
- Erzeugt immer Ausgabe einer fixen Länge
- Für den Hashwert darf kein anderer Ausgangstext gefunden werden, als der gehashte.
- Es dürfen nicht mehrere verschiedene Ausgangstexte gefunden werden, die denselben Hashwert erzeugen.
- Auch geringe Änderungen im Ausgangstext müssen signifikante Änderungen im Hashwert erzeugen.
- Hashwert kann für beliebige Ausgangsdatei einfach und schnell errechnet werden

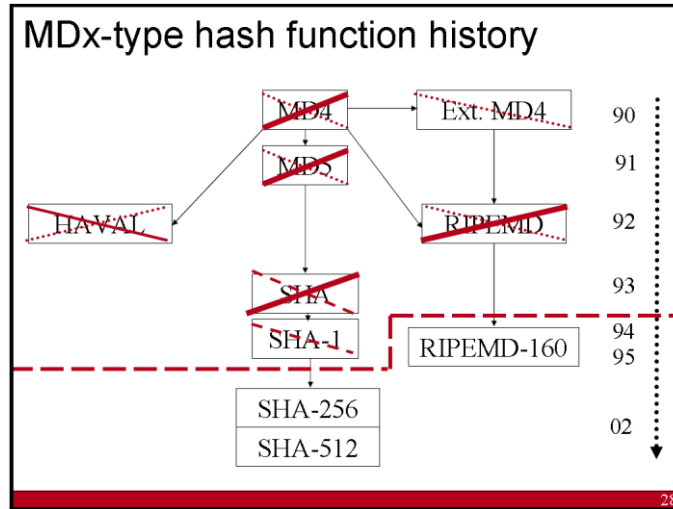
MD5

- Message Digest #5, von Ron Rivest
- Eingabe in 512 Bit Blöcken (kürzere Nachrichten werden aufgefüllt)
- Hashwert: 128 Bit (zu klein für die meisten Anwendungen!)
- 4 Runden
- Theoretische Angriffe waren erfolgreich => dies hat keine Auswirkungen auf praktische Anwendungen
- Dieses Verfahren soll nicht mehr eingesetzt werden! (ist aber noch in vielen Standards vorhanden, z.B. IPSec)

SHA-3 (Secure Hash Algorithm)

- 2012 wurde Keccak vom NIST zum Standard ausgewählt.
- Hashwert: 224 Bit, 256 Bit, 384 Bit, 512 Bit
- Einstellbare Rundenzahl
- Einstellbare Wortlänge

Geschichte der Hashfunktionen



02

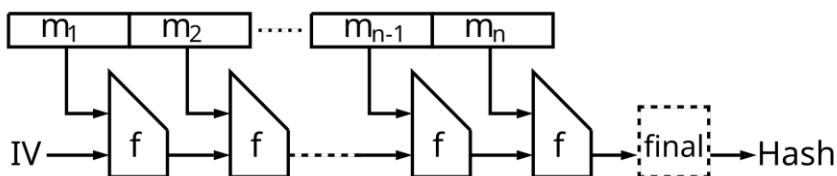
Merkle-Damgård-Construction

MAC: Message Authentication Code

- Die Merkle-Damgård-Konstruktion ist eine weit verbreitete Technik in der Kryptographie zur **Entwicklung kryptographischer Hashfunktionen**.
- Sie bietet eine Methode zur **Konstruktion einer Hashfunktion, die eine Eingabe beliebiger Länge entgegennimmt und einen Hashwert fester Größe erzeugt**.
- Die Konstruktion wurde unabhängig voneinander von Ralph Merkle und Ivan Damgård Ende der 1970er Jahre vorgeschlagen.
- Sie wurde in vielen bekannten Hashfunktionen verwendet, darunter MD5, SHA-1 und die SHA-2-Familie (SHA-224, SHA-256, SHA-384, SHA-512 usw.).

Vorgehensweise

Aus den **Nachrichtenblöcken** ($m_1 \dots m_n$) wird durch wiederholte Anwendung der **Kompressionsfunktion** f der Hashwert erzeugt. Die 1. Iteration beginnt mit einem **Initialisierungsvektor** IV .



MAC: Message Authentication Code

- **Padding:** Die Eingabebotschaft wird in Blöcke fester Größe unterteilt. Falls die Länge der Nachricht nicht ein Vielfaches der Blockgröße ist, wird Padding zum letzten Block hinzugefügt, um ihn zu vervollständigen.
- **Initialization:** Die Hash-Funktion wird mit einem Initialwert initialisiert, der als Initialisierungsvektor (IV) bezeichnet wird. Dieser IV ist ein Wert fester Größe, der üblicherweise vom Designer der Hash-Funktion ausgewählt wird.
- **Kompression:** Das Herzstück der Merkle-Damgård-Konstruktion ist eine Kompressionsfunktion. Sie nimmt einen Block fester Größe und den aktuellen Hash-Wert als Eingabe entgegen und erzeugt einen neuen Hash-Wert. Die Kompressionsfunktion ist darauf ausgelegt, recheneffizient zu sein und sollte Eigenschaften haben, die es erschweren, die Eingabe aus der Ausgabe umzukehren (reverse-engineering).
- **Iteration:** Die Kompressionsfunktion wird iterativ auf jeden Block der Eingabebotschaft angewendet, wobei der aktuelle Hash-Wert als Eingabe für die Kompressionsfunktion verwendet wird. Nach jeder Iteration wird der Hash-Wert aktualisiert.
- **Finalisierung:** Nachdem alle Blöcke verarbeitet wurden, wird der endgültige Hash-Wert erhalten. In vielen Hash-Funktionen werden zusätzliche Operationen auf dem endgültigen Hash-Wert durchgeführt, wie zum Beispiel bitweise Operationen oder Abschneidungen, um die gewünschte Ausgabegröße zu erzeugen.



[Quelle: <https://www.hashgenerator.de/hintergrundinformationen-zu-hashfunktionen/merkle-damgard-konstruktion/> - letzter Abruf 20.6.2026]

tgm | Technologisches Gewerbemuseum | Höhere technische Bundes-Lehr- und Versuchsanstalt

25

25

Anwendungsfelder

Sicherheitsgrundlage

- basiert auf sicherer Kompressionsfunktion
- beruht auf Schwierigkeit von:
 - Kollisionen (zwei Inputs → gleicher Hash)
 - Pre-Images (Eingabe zu gegebenem Hash finden)

Typische Anwendungen

- Datenintegrität (Dateiprüfung, Checksummen)
- Digitale Signaturen
- Passwortspeicherung (Hashing)
- Zertifizierungsstellen (PKI)

Vorteile

- effiziente Verarbeitung großer Datenmengen
- feste Hashlänge unabhängig von Eingabegröße
- eindeutige Identifikation von Daten

Bedeutung

- zentrale Grundlage moderner Kryptographie
- entscheidend für Sicherheit digitaler Systeme



[Quelle: <https://www.hashgenerator.de/hintergrundinformationen-zu-hashfunktionen/merkle-damgard-konstruktion/> - letzter Abruf 20.6.2026]

tgm | Technologisches Gewerbemuseum | Höhere technische Bundes-Lehr- und Versuchsanstalt

26

26

Angriffsszenarien

Kollisionsangriffe

- Zwei unterschiedliche Eingaben → gleicher Hashwert
- Gefahr: Manipulation von Daten bleibt unentdeckt
- Beispiel: gefälschte Datei mit identischem Hash

Präbildangriffe

- Gegebener Hash → passende Eingabe finden
- Gefahr: Kompromittierung von Signaturen oder Authentifizierung

Gegenmaßnahmen

- Verwendung kryptographisch sicherer Hashfunktionen
- regelmäßige Anpassung an aktuelle Standards
- Einsatz moderner Verfahren (z. B. SHA-2 / SHA-3)

03

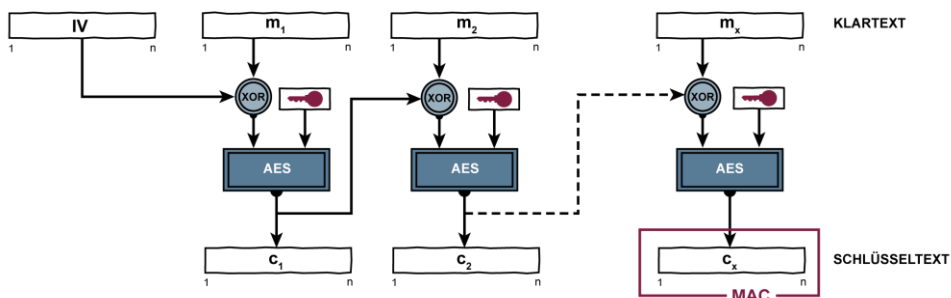
MAC

MAC: Message Authentication Code

- Ein Message Authentication Code oder MAC ist eine Einweg-Hashfunktion, die einen Schlüssel enthält, mit nur diesem man den Hashwert verifizieren kann.
- Damit kann man Authentizität ohne Geheimhaltung erreichen.
- Mit Hilfe von MACs können mehrere Benutzer ihre Dateien authentifizieren und einzelne Benutzer können mit MACs überprüfen, ob ihre Dateien verändert wurden, z.B. von Malware.
- Im Gegensatz zu Einweg-Hashfunktionen ist der Schlüssel des MACs zur Berechnung des Hashwertes nur dem Benutzer bekannt und der Hashwert kann nicht unbemerkt verändert werden.
- Eine einfache Möglichkeit zur Umwandlung einer Einweg-Hashfunktion in einen MAC besteht darin, den Hashwert mit einem symmetrischen Algorithmus zu verschlüsseln.
- Jeder MAC kann in eine Einweg-Hashfunktion umgewandelt werden, indem man den Schlüssel veröffentlicht.

CBC MAC

- CBC MAC, weit verbreiteter Standard [NIS85].
- Die Idee ist die Verwendung von DES (oder auch andere wie AES) im CBC-Modus und die anschließende Verwendung des letzten Blocks des Ciphertextes als Prüfsumme.
- Diese Verfahren wird oft in der Bankenwelt verwendet.



HMAC

- HMAC (Keyed-Hashing for Message Authentication) Randbedingungen: (Internet-Standard (RFC 2104), z.B. IPSec)
- Die Geschwindigkeit der Hashfunktion soll nur wesentlich verlangsamt werden!
- Das Verfahren soll mit möglichst vielen Hashfunktionen zusammenarbeiten, ohne dass diese modifiziert werden müssen.
- Die Sicherheit der Hashfunktion darf durch die Manipulation mit geheimen Schlüsseln nicht verringert werden.

HMAC

HMAC = KH(K XOR opad, H(K XOR ipad, M))

- ipad (inner padding) = 0x36, 0x36, 0x36, ... (gleiche Länge wie die Blocklänge der Hashfunktion)
- opad (outer padding) = 0x5c, 0x5c, 0x5c, ... (gleiche Länge wie die Blocklänge der Hashfunktion)
- K = geheimer Schlüssel
- M = Input (Nachricht)
- XOR = bitweise modulo 2 addieren
- H = „Keyed-Hashing for Message Authentication“-Verfahren – HMAC-Verfahren

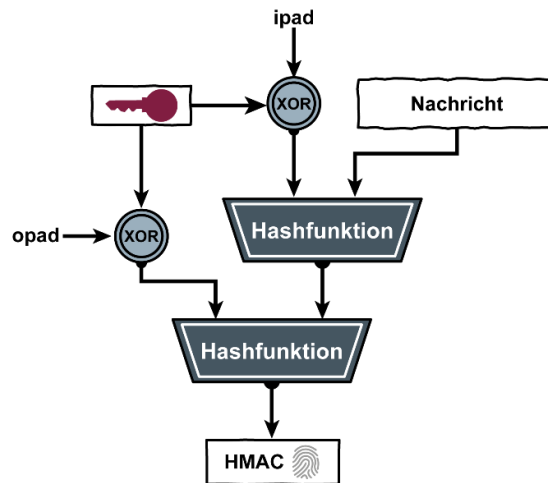
HMAC

- Die Felder ipad und opad haben eine Länge, die der Blockgröße B der eingesetzten Hashfunktion entspricht (64 Bytes bei SHA-1 und RIPEMD).
- Der Schlüssel K wird durch das Anhängen von Nullen ebenfalls auf die Länge B gebracht.
- Verknüpfe den auf die Länge B gebrachten geheimen Schlüssel K mittels XOR mit dem Feld ipad.
- Stelle das Ergebnis dieser Operation vor die Nachricht und berechne mit der Hashfunktion den Hashwert aus diesem Input.
- Der Hashwert hat die Länge L (16 Byte bei SHA-1 und RIPEMD).

HMAC

- Verknüpfe den auf die Länge B gebrachten geheimen Schlüssel K mittels XOR mit dem Feld opad.
- Stelle das Ergebnis dieser Operation (Länge B) vor den Hashwert (Länge L) und berechne mit der Hashfunktion den HMAC-Hashwert.
- Der HMAC-Hashwert hat die Länge L (16 Byte bei SHA-1 und RIPEMD).

HMAC



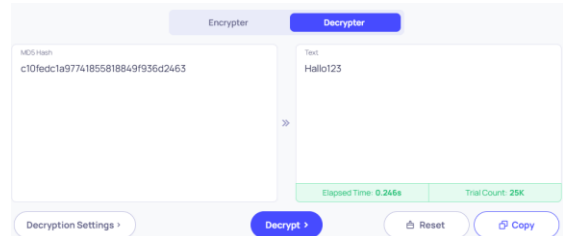
04

Regenbogentabelle

Das Risiko von (alten) Hash-Funktionen!

Beispiel: <https://10015.io/tools/md5-encrypt-decrypt>

- **Schlechte Hash-Funktionen sind manipulierbar und sogar dechiffrierbar.** Es lassen sich keine ganzen Textdokumente wiederherstellen, aber Passwörter wie „Hallo123“ sind in Sekunden decodiert.
- Du kannst das ganze gerne einmal selber ausprobieren.
 - Passwort: Hallo123
 - Hash: c10fedc1a97741855818849f936d2463
 - Verschlüsselt mit **MD5** (Message-Digest Algorithm 5).
- Gib in **Google Md5 Hash entschlüsseln** ein und aus dieser willkürlichen Zeichenfolge wird wieder der Anfangs Text.
- Der Algorithmus **MD5 war jahrelang eine Standardverschlüsselung im Internet**, mittlerweile wurde er von sichereren Algorithmen abgelöst und ersetzt.



tgm [Quelle: <https://10015.io/tools/md5-encrypt-decrypt> - letzter Abruf 14.4.2026]

tgm | Technologisches Gewerbemuseum | Höhere technische Bundes-Lehr- und Versuchsanstalt

37

37

Regenbogentabellen

<https://freerainbowtables.com/>

Free Rainbow Tables
Distributed Rainbow Table Project

The goal of FreeRainbowTables.com is to prove the insecurity of using simple hash routines to protect valuable passwords, and force developers to use [more secure methods](#). By [distributing](#) the generation of rainbow chains, we can generate [HUGE rainbow tables](#) that are able to crack [any password](#) that ever seen before. Furthermore, we are also improving the rainbow table technology, making them even [smaller](#) and [faster](#) than rainbow tables found elsewhere, and the best thing is, those tables are freely available!

Character set and password length Hover your mouse over the below for more information	NTLM 4 TB	SHA-1 ¹ and MySQLSHA1 3 TB	MD5 4.3 TB	LM 398 GB	Half LM challenge 18 GB
all-space#1-7 ²				34 GB: 0:123	18 GB: 0:123
alpha#1-1,loweralpha#5-5,loweralpha-numeric#2-2,numeric#1-3	362 GB: 0:123		362 GB: 0:123		
alpha-space#1-9	35 GB: 0:123		23 GB: 0:123		
lm-ft-cp#37-850#1-7				364 GB: 0:123	
loweralpha#1-10		179 GB: 0:123	179 GB: 0:123		
loweralpha#7-7,numeric#1-3	26 GB: 0:123		26 GB: 0:123		
loweralpha-numeric#1-10	587 GB: 0:81624	587 GB: 0:81624	588 GB: 0:81624		
loweralpha-numeric-space#1-8	15 GB: 0:123	17 GB: 0:123	16 GB: 0:123		
loweralpha-numeric-space#1-9		108 GB: 0:123	108 GB: 0:123		
loweralpha-numeric-symbol32-space#1-7	33 GB: 0:123	33 GB: 0:123	33 GB: 0:123		
loweralpha-numeric-symbol32-space#1-8	428 GB: 0:123	427 GB: 0:123	425 GB: 0:123		
loweralpha-space#1-9	35 GB: 0:123	38 GB: 0:123	35 GB: 0:123		
mixalpha-numeric#1-8	274 GB: 0:123				
mixalpha-numeric#1-9	1 TB: 0:163248	504 GB: 0:16	1 TB: 0:163248		
mixalpha-numeric-space#1-4	17 GB: 0:123		17 GB: 0:123		
mixalpha-numeric-space#1-7			207 GB: 0:123		
mixalpha-numeric-symbol32-space#1-7 ³	86 GB: 0:123	86 GB: 0:123	86 GB: 0:123		
mixalpha-numeric-symbol32-space#1-8 ²	1 TB: 0:8162432	1 TB: 0:81624	1 TB: 0:8162432		
numeric#1-12		5 GB: 0:123			
numeric#1-14			90 GB: 0:123		

tgm [Quelle: <https://freerainbowtables.com/> - letzter Abruf 14.4.2026]

tgm | Technologisches Gewerbemuseum | Höhere technische Bundes-Lehr- und Versuchsanstalt

38

38

BACKUP

39

IT-SICHERHEIT / SEMESTER 5 UND 6

Glossar

- **Avalanche-Effekt:** Auch Lawinen-Effekt. Ein Prinzip bei Hash-Algorithmen, das besagt, dass selbst minimale Änderungen an den Eingangsdaten einen völlig neuen Hash-Wert erzeugen.
- **Blockchain:** Eine dezentrale Datenbank-Technologie, die mithilfe von Hash-Werten und Kryptografie Daten in Blöcken speichert und diese zu einer unveränderlichen, transparenten Kette verknüpft. Typische Anwendungen sind z. B. Kryptowährungen und fälschungssichere Nachweise.
- **Datenintegrität:** Der Zustand, in dem Daten vollständig, unverändert und zuverlässig sind. Sie stellt sicher, dass Informationen während der Speicherung, Übertragung oder Verarbeitung korrekt bleiben und nicht manipuliert wurden.
- **Digitale Signatur:** Eine elektronische Signatur, die mithilfe kryptografischer Verfahren die Echtheit und Integrität eines Dokuments garantiert. Sie verbindet den Hash-Wert des Dokuments mit der Identität des Unterzeichners.
- **Hashfunktion:** Das Verfahren, mit dem der Hash gebildet wird.
- **Hashwert:** Das Ergebnis des Hashings.

40

Glossar

- **Hash-Algorithmus:** Ein konkretes Verfahren, das die mathematische Hashfunktion umsetzt und Daten beliebiger Grösse in einen festen, einzigartigen Wert umwandelt. Beispiele sind SHA-256 oder SHA3-256.
- **Hash-Wert:** Ein Hash-Wert ist ein digitaler Fingerabdruck, der durch einen Hash-Algorithmus aus den Eingangsdaten generiert wird. Er ist einzigartig für die jeweiligen Daten und ermöglicht es, deren Integrität sicherzustellen. Selbst kleinste Änderungen an den Daten führen zu einem komplett neuen Hash-Wert (Avalanche-Effekt). Ein Hash-Wert hat eine feste Länge, unabhängig von der Größe der Eingangsdaten (z. B. 256 Bit bei SHA-256), und ist durch die Einweg-Funktion nicht zurückrechenbar. Hash-Werte werden in digitalen Signaturen, Blockchain und zur Passwortsicherung verwendet und garantieren, dass Daten unverändert bleiben.
- **Kollision:** wenn aus zwei Eingaben der selbe Hashwert gebildet wird.
- **Salt:** für Passwort-Hashing benutzt, wird vor dem Hashing an die Eingabe angehängt, um Angriffe per vorberechneten Hash-Tabellen zu verhindern.
- **Rounds:** für Passwort-Hashing benutzt, Hash wird mehrfach berechnet, um Brute-Force-Angriffe zu verlangsamen